Department of Electronics and Electrical Communication Engineering
Indian Institute of Technology Kharagpur

Embedded Systems Laboratory

EC39202

Experiment 3: Real Time Clock with Stop Watch



Soumadipta Majumder
23EC30055

Jai Raj
23EC10031

Group 5

3 April, 2026

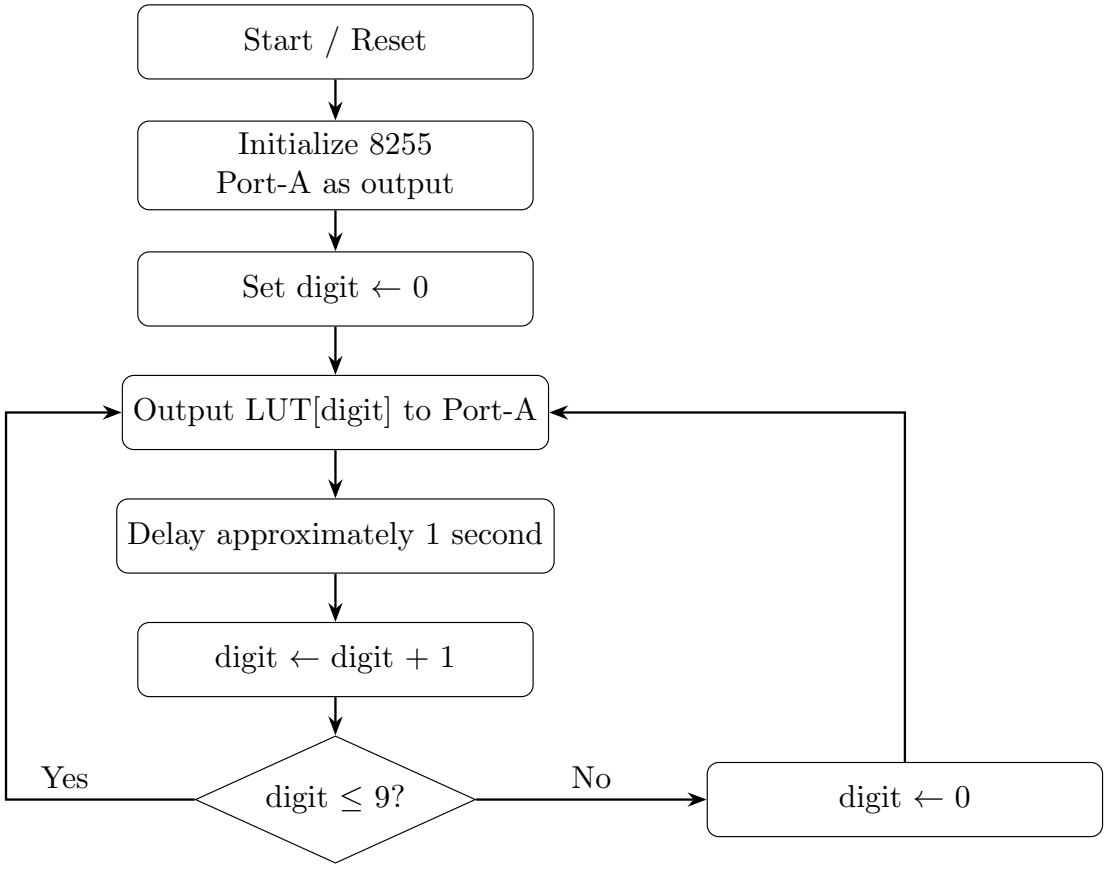
1. Problem Statement

Objective:

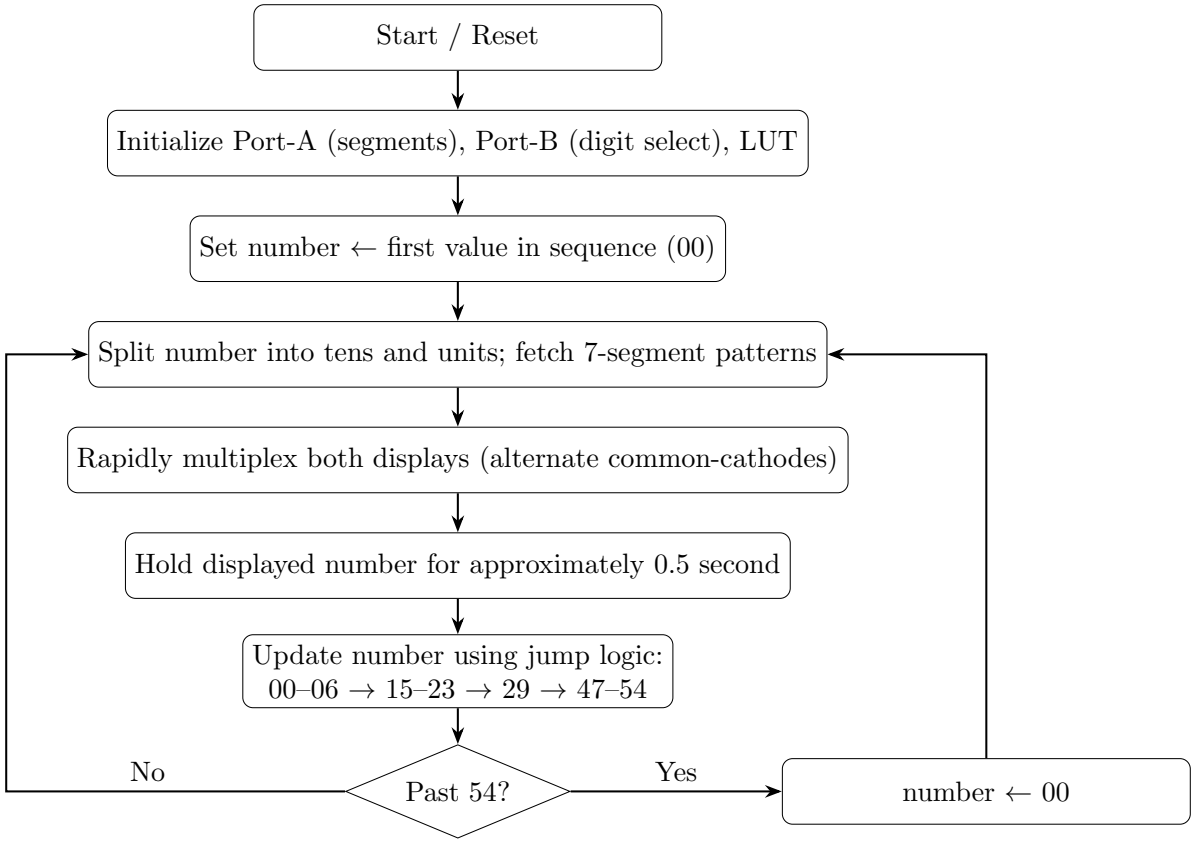
1. Interface common-cathode 7-segment display(s) with the 8051 trainer kit through 8255 PPI and 74245 line drivers.
2. Display digits **0 to 9** continuously on one 7-segment display with approximately **1 s** hold per digit.
3. Display the discontinuous sequence **00–06, 15–23, 29, 47–54** on two multiplexed 7-segment displays with approximately **0.5 s** hold per number.
4. Design and implement a dual-mode system: **normal real-time clock** and **stopwatch** (hh-mm-ss), selectable using a mode switch.
5. Use timer interrupt driven time-base so that in stopwatch mode, both the normal clock and stopwatch counters continue to update correctly.

2. Flow Chart

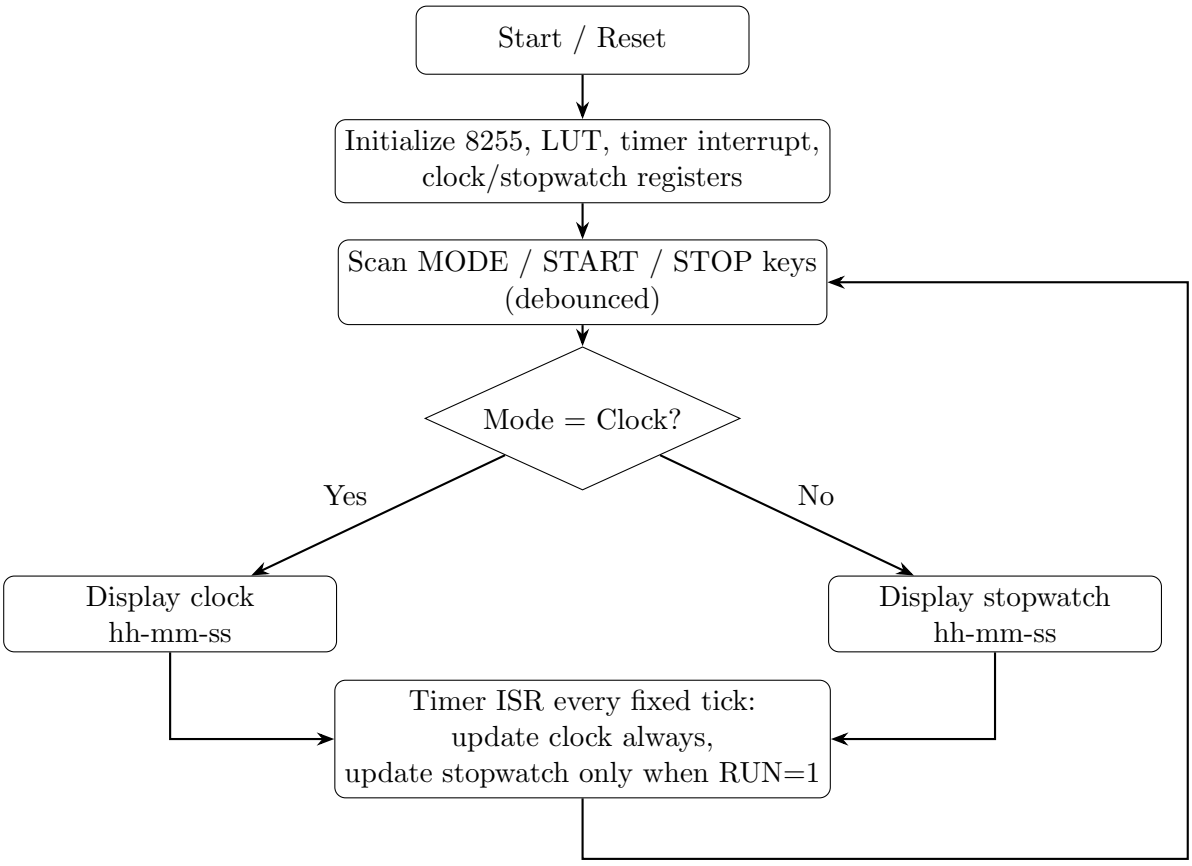
Part (a): Single display 0 to 9



Part (b): Two-display custom sequence



Part (c): Real-time clock with stopwatch



- Part (a) and Part (b) are realized using polling + software delays.
- Part (c) uses timer interrupt for accurate 1-second time roll-over.

3. Circuit Diagram, Pin Description, and Components

Components used:

- 8051 microcontroller trainer kit
- 8255 Programmable Peripheral Interface (PPI)
- IC 74245 bus transceivers (segment and digit driving stages)
- IC 74373 latch (address/data demultiplex support on trainer)
- One / two / six common-cathode 7-segment display modules (as per part)
- Current-limiting resistors (220Ω–470Ω) for each segment line

- Push-buttons for Mode, Start/Reset and Stop inputs (Part c)
- Breadboard, FRC cable, jumpers, regulated +5V supply
- External DC supply and jumper wires
- **Softwares Used**: Keil μVision, Win51

Circuit diagrams: Pin usage (as per program):

- **8255 Port A (0E800H):** Segment data bus (a,b,c,d,e,f,g,dp) through 74245.
- **8255 Port B (0E801H):** Digit select / common-cathode enable lines through second 74245 (multiplexing).
- **8255 Port C (0E802H):** Switch input lines in Part (c): Mode, Start/Reset, Stop.
- **8255 Control register (0E803H):** I/O mode configuration (Mode 0).
- **8051 Timer-0:** Periodic interrupt source for timekeeping in Part (c).

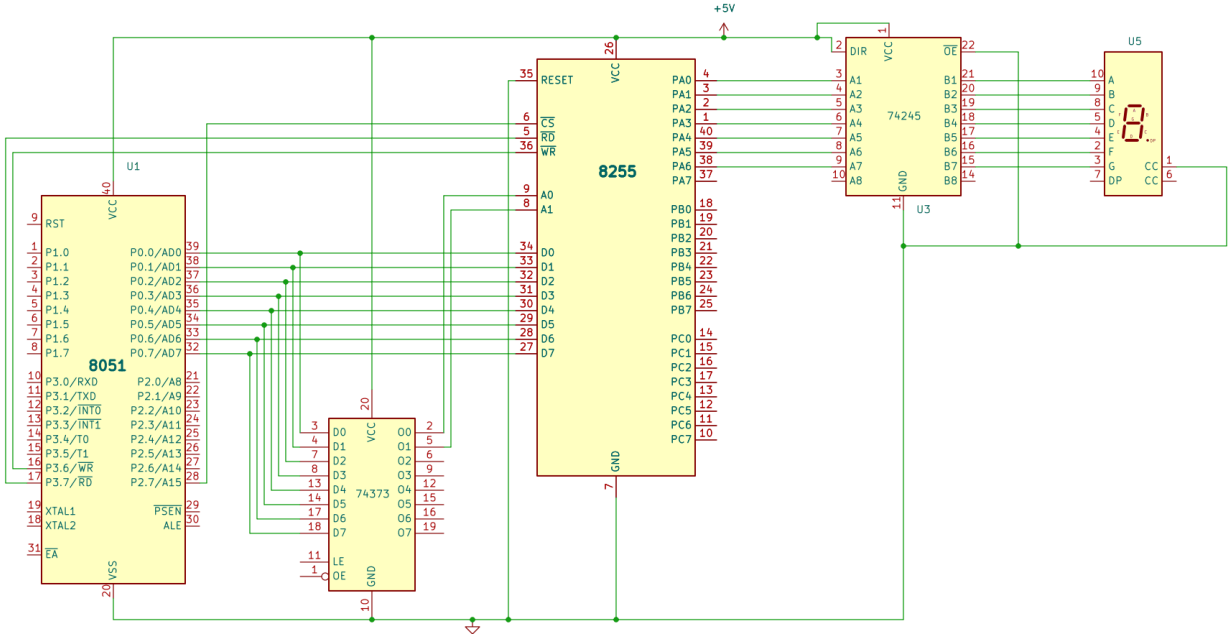


Figure 1: Part (a): Single 7-segment display interface through 8255 and 74245.

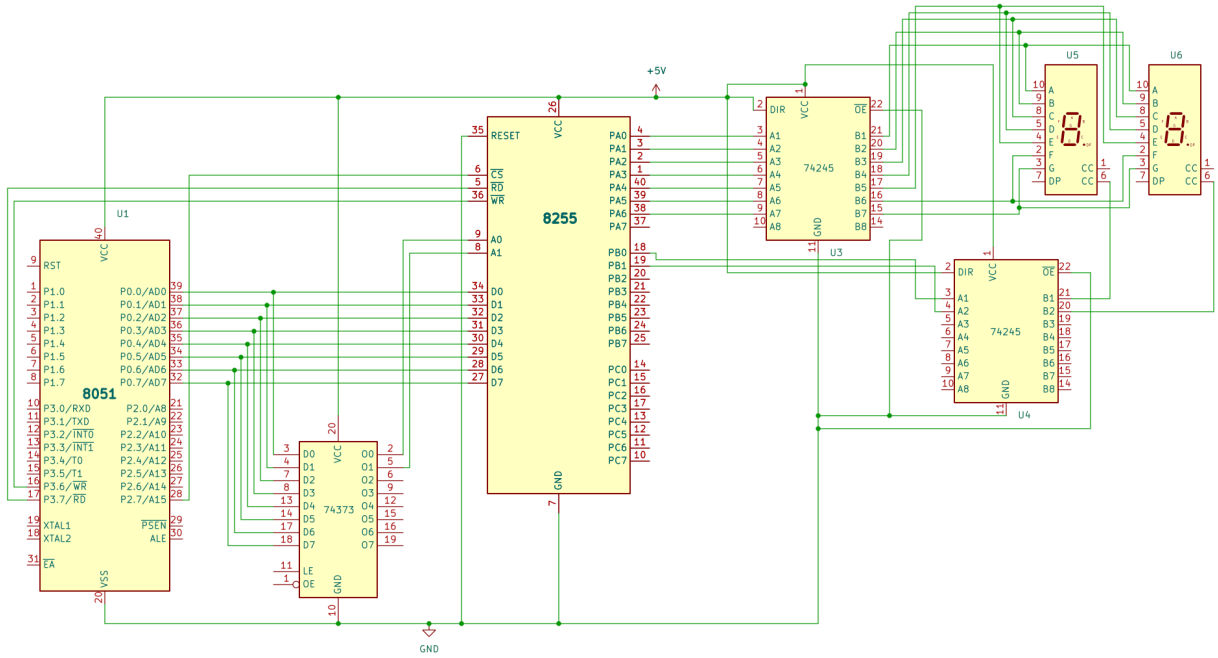


Figure 2: Part (b): Two-display multiplexed interface (shared segments, independent common-cathodes).

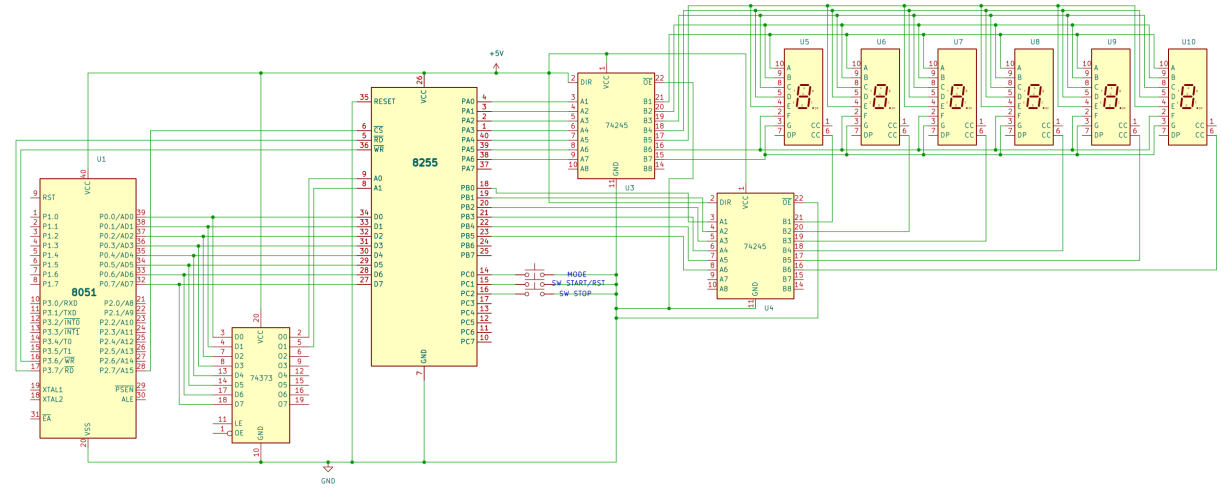


Figure 3: Part (c): Clock + stopwatch hardware with 6-digit multiplexed display and mode/control switches.

External memory mapping:

- `0E800H` → 8255 Port A

- `0E801H` → 8255 Port B
- `0E802H` → 8255 Port C
- `0E803H` → 8255 Control Word Register
- `9000H--9002H` → Clock BCD values (HH, MM, SS)
- `9003H--9005H` → Stopwatch BCD values (HH, MM, SS)

Connection details:

- Segment lines are connected in parallel for all displays; each segment has individual series resistor.
- Each display's common cathode is driven independently for multiplex scanning.
- Switches are wired as active-low inputs (pressed ⇒ logic 0) and debounced in software.
- 74245 outputs provide sufficient current drive and protect MCU/PPI lines.
- Ground of 8051 kit and external supply tied together (mandatory common reference).

4. Pseudo-code / Algorithm

Part (a): Single display 0 to 9

1. Configure 8255 Port-A as output.
2. Load lookup table for 7-segment codes for digits 0 to 9.
3. For each digit from 0 to 9:
   - Send segment code to Port-A.
   - Wait approximately 1 second.
4. Repeat forever.

Part (b): Two-display custom sequence

1. Configure Port-A for segment data and Port-B for digit enable signals.
2. Set number variable to first value in sequence.
3. Convert number into tens and units digits.
4. Map each digit through LUT to 7-segment pattern.

5. Multiplex both displays rapidly (alternate enable of each common cathode).
6. Hold each number for approximately 0.5 second.
7. Increment number and jump across ranges as:
    - 00–06, then 15–23, then 29, then 47–54, then back to 00.
8. Repeat forever.

Part (c): Real-time clock with stopwatch

1. Initialize 8255 ports, timer interrupt, and BCD registers for clock and stopwatch.
2. In main loop, read Mode/Start/Stop buttons (active-low with debounce).
3. If Mode = Clock, display clock registers; else display stopwatch registers.
4. Timer ISR executes at fixed tick:
    - Accumulate ticks to 1 second.
    - Update normal clock every second.
    - Update stopwatch every second only if RUN flag is set.
5. Start button (in stopwatch mode): reset stopwatch to 00:00:00 and set RUN.
6. Stop button (in stopwatch mode): clear RUN.
7. Continue multiplex display refresh continuously.

5. Actual Code (8051 Assembly)

(a) Single display, 0–9 with 1 s delay

Listing 1: Part (a) assembly source

```
ORG 8000H

ZERO EQU 3FH
ONE EQU 6H
TWO EQU 5BH
THREE EQU 4FH
FOUR EQU 66H
FIVE EQU 6DH
SIX EQU 7DH
SEVEN EQU 7H
EIGHT EQU 7FH
NINE EQU 6FH

MOV DPTR , #0E803H
```

```
MOV A , #80H
MOVX @DPTR , A

LOOP:
        MOV DPTR , #0E800H
        MOV A , #ZERO
        MOVX @DPTR , A
        ACALL DELAY
        MOV A , #ONE
        MOVX @DPTR , A
        ACALL DELAY
        MOV A , #TWO
        MOVX @DPTR , A
        ACALL DELAY
        MOV A , #THREE
        MOVX @DPTR , A
        ACALL DELAY
        MOV A , #FOUR
        MOVX @DPTR , A
        ACALL DELAY
        MOV A , #FIVE
        MOVX @DPTR , A
        ACALL DELAY
        MOV A , #SIX
        MOVX @DPTR , A
        ACALL DELAY
        MOV A , #SEVEN
        MOVX @DPTR , A
        ACALL DELAY
        MOV A , #EIGHT
        MOVX @DPTR , A
        ACALL DELAY
        MOV A , #NINE
        MOVX @DPTR , A
        ACALL DELAY
        SJMP LOOP

HERE: SJMP HERE



DELAY:
MOV R2 ,#10
D3: MOV R1 ,#200
D2: MOV R0 ,#255
D1: DJNZ R0,D1
DJNZ R1 ,D2
DJNZ R2 ,D3
RET


END
```

(b) Two displays, sequence 00–06, 15–23, 29, 47–54

Listing 2: Part (b) assembly source

```
ORG 8000H


MOV R5 , #00 //LOWERLIMIT
MOV DPTR , #0E803H
MOV A , #80H
MOVX @DPTR , A
MOV B ,#10


LOOP2:
        MOV R3 ,#30
        ACALL READ_INC
LOOP1:

        MOV DPTR , #0E800H
        MOV A , R7
        MOVX @DPTR , A
        MOV DPTR , #0E801H
        MOV A , #11111110B
        MOVX @DPTR , A
        ACALL DELAY_2

        MOV DPTR , #0E800H
        MOV A , R6
        MOVX @DPTR , A
        MOV DPTR , #0E801H
        MOV A , #11111101B
        MOVX @DPTR , A
        ACALL DELAY_2
        DJNZ R3 , LOOP1

        MOV A , R5
        CJNE A , #07 , LOOP2
        MOV R5 , #15
        SJMP LOOP4

LOOP4:
        MOV R3 ,#30
        ACALL READ_INC
LOOP5:

        MOV DPTR , #0E800H
        MOV A , R7
        MOVX @DPTR , A
        MOV DPTR , #0E801H
        MOV A , #11111011B
        MOVX @DPTR , A
        ACALL DELAY_2

        MOV DPTR , #0E800H
```

```
        MOV A , R6
        MOVX @DPTR , A
        MOV DPTR , #0E801H
        MOV A , #11110111B
        MOVX @DPTR , A
        ACALL DELAY_2
        DJNZ R3 , LOOP5

        MOV A , R5
        CJNE A , #24 , LOOP4
        MOV R5 , #29
        SJMP LOOP6


LOOP6:
        MOV R3 ,#30
        ACALL READ_INC
LOOP7:

        MOV DPTR , #0E800H
        MOV A , R7
        MOVX @DPTR , A
        MOV DPTR , #0E801H
        MOV A , #11101111B
        MOVX @DPTR , A
        ACALL DELAY_2

        MOV DPTR , #0E800H
        MOV A , R6
        MOVX @DPTR , A
        MOV DPTR , #0E801H
        MOV A , #11011111B
        MOVX @DPTR , A
        ACALL DELAY_2
        DJNZ R3 , LOOP7

        MOV A , R5
        CJNE A , #45 , LOOP6
        MOV R5 , #47
        SJMP LOOP9

LOOP9:
        MOV R3 ,#30
        ACALL READ_INC
LOOP10:

        MOV DPTR , #0E800H
        MOV A , R7
        MOVX @DPTR , A
        MOV DPTR , #0E801H
        MOV A , #11111110B
        MOVX @DPTR , A
        ACALL DELAY_2

```

```
        MOV DPTR , #0E800H
        MOV A , R6
        MOVX @DPTR , A
        MOV DPTR , #0E801H
        MOV A , #11111101B
        MOVX @DPTR , A
        ACALL DELAY_2
        DJNZ R3 , LOOP10

        MOV A , R5
        CJNE A , #55 , LOOP9
        MOV R5 , #00
        LJMP LOOP2



READ_INC:
        MOV A ,R5
        DIV AB
        MOV R6 , A
        MOV R7 , B
        MOV B ,#10

        MOV DPH , #90H
        MOV DPL , R6
        MOVX A,@DPTR
        MOV R6,A

        MOV DPH , #90H
        MOV DPL , R7
        MOVX A,@DPTR
        MOV R7,A

        INC R5
        RET

DELAY:
MOV R2 ,#10
D3: MOV R1 ,#200
D2: MOV R0 ,#255
D1: DJNZ R0,D1
DJNZ R1 ,D2
DJNZ R2 ,D3
RET

DELAY_2:
MOV R2 ,#4
D6: MOV R1 ,#5
D5: MOV R0 ,#255
D4: DJNZ R0,D4
DJNZ R1 ,D5
DJNZ R2 ,D6
RET

```

```
DELAY_3:
MOV R2 ,#255
D7:DJNZ R2,D7
RET


HERE:SJMP HERE
END
```

(c) Real-time clock with stopwatch mode

Listing 3: Part (c) assembly source

```
PORTA EQU 0E800H
PORTB EQU 0E801H
PORTC EQU 0E802H
CTRL EQU 0E803H

CLK_HR EQU 9000H
CLK_MIN EQU 9001H
CLK_SEC EQU 9002H

SW_HR EQU 9003H
SW_MIN EQU 9004H
SW_SEC EQU 9005H

MODE_BIT EQU 20H.0
RUN_BIT EQU 20H.1

ORG 0FFF0H
    LJMP TIMER0_ISR

ORG 8100H
INIT:
    MOV DPTR, #CTRL
    MOV A, #89H
    MOVX @DPTR, A

    MOV A, #05H
    MOV DPTR, #CLK_HR
    MOVX @DPTR, A
        MOV A, #00H
    MOV DPTR, #CLK_MIN
    MOVX @DPTR, A
        MOV A, #00H
    MOV DPTR, #CLK_SEC
    MOVX @DPTR, A

        MOV A, #00H
    MOV DPTR, #SW_HR
    MOVX @DPTR, A
    MOV DPTR, #SW_MIN
    MOVX @DPTR, A
    MOV DPTR, #SW_SEC
```

```
    MOVX @DPTR, A

    CLR MODE_BIT
    CLR RUN_BIT

    MOV TMOD, #01H
    MOV TH0, #4BH
    MOV TL0, #27H
    MOV R2, #20

    SETB ET0
    SETB EA
    SETB TR0

MAIN_LOOP:
    MOV DPTR, #PORTC
    MOVX A, @DPTR
    MOV R5, A

    ; --- ACTIVE LOW: Trigger when wire touches Ground ---
    JNB ACC.0, TOGGLE_MODE
    SJMP CHK_START
TOGGLE_MODE:
    CPL MODE_BIT
    ACALL DEBOUNCE

CHK_START:
    MOV A, R5
    JNB ACC.1, START_SW
    SJMP CHK_STOP
START_SW:
    JNB MODE_BIT, CHK_STOP ; Ignore Start if looking at the Clock
    SETB RUN_BIT
    MOV A, #00H
    MOV DPTR, #SW_HR
    MOVX @DPTR, A
    MOV DPTR, #SW_MIN
    MOVX @DPTR, A
    MOV DPTR, #SW_SEC
    MOVX @DPTR, A
    ACALL DEBOUNCE

CHK_STOP:
    MOV A, R5
    JNB ACC.2, STOP_SW
    SJMP DISP_ROUTINE
STOP_SW:
    JNB MODE_BIT, DISP_ROUTINE ; Ignore Stop if looking at the Clock
    CLR RUN_BIT
    ACALL DEBOUNCE

DISP_ROUTINE:
    JNB MODE_BIT, DISP_CLOCK
    MOV R0, #90H
```

```
    MOV R1, #03H
    SJMP MULTIPLEX_OUT
DISP_CLOCK:
    MOV R0, #90H
    MOV R1, #00H

MULTIPLEX_OUT:
    ACALL MULTIPLEX_DISPLAY
    SJMP MAIN_LOOP

TIMER0_ISR:
    PUSH ACC
    PUSH PSW ; Protect Carry Flag
    PUSH DPH
    PUSH DPL

    MOV TH0, #3CH
    MOV TL0, #0B0H

    DJNZ R2, ISR_END
    MOV R2, #20

    ACALL UPDATE_CLOCK

    JNB RUN_BIT, ISR_END
    ACALL UPDATE_SW

ISR_END:
    POP DPL
    POP DPH
    POP PSW
    POP ACC
    RETI

UPDATE_CLOCK:
    MOV DPTR, #CLK_SEC
    MOVX A, @DPTR
    CLR C ; Clear carry before math
    ADD A, #01H
    DA A
    MOVX @DPTR, A
    CJNE A, #60H, CLK_RET
    MOV A, #00H
    MOVX @DPTR, A

    MOV DPTR, #CLK_MIN
    MOVX A, @DPTR
    CLR C
    ADD A, #01H
    DA A
    MOVX @DPTR, A
    CJNE A, #60H, CLK_RET
    MOV A, #00H
    MOVX @DPTR, A
```

```
150
151     MOV DPTR, #CLK_HR
152     MOVX A, @DPTR
153     CLR C
154     ADD A, #01H
155     DA A
156     MOVX @DPTR, A
157     CJNE A, #24H, CLK_RET
158     MOV A, #00H
159     MOVX @DPTR, A
160
161 CLK_RET:
162     RET
163
164 UPDATE_SW:
165     MOV DPTR, #SW_SEC
166     MOVX A, @DPTR
167     CLR C
168     ADD A, #01H
169     DA A
170     MOVX @DPTR, A
171     CJNE A, #60H, SW_RET
172     MOV A, #00H
173     MOVX @DPTR, A
174
175     MOV DPTR, #SW_MIN
176     MOVX A, @DPTR
177     CLR C
178     ADD A, #01H
179     DA A
180     MOVX @DPTR, A
181     CJNE A, #60H, SW_RET
182     MOV A, #00H
183     MOVX @DPTR, A
184
185     MOV DPTR, #SW_HR
186     MOVX A, @DPTR
187     CLR C
188     ADD A, #01H
189     DA A
190     MOVX @DPTR, A
191     CJNE A, #24H, SW_RET
192     MOV A, #00H
193     MOVX @DPTR, A
194
195 SW_RET:
196     RET
197
198 MULTIPLEX_DISPLAY:
199     MOV DPH, R0
200     MOV DPL, R1
201     MOVX A, @DPTR
202     MOV R5, A
203
```

```
    SWAP A
    ANL A, #0FH
    ACALL GET_LUT
    MOV DPTR, #PORTA
    MOVX @DPTR, A
    MOV DPTR, #PORTB
    MOV A, #0DFH
    MOVX @DPTR, A
    ACALL SHORT_DELAY
    MOV A, #0FFH
    MOVX @DPTR, A

    MOV A, R5
    ANL A, #0FH
    ACALL GET_LUT
    MOV DPTR, #PORTA
    MOVX @DPTR, A
    MOV DPTR, #PORTB
    MOV A, #0EFH
    MOVX @DPTR, A
    ACALL SHORT_DELAY
    MOV A, #0FFH
    MOVX @DPTR, A

    INC R1
    MOV DPH, R0
    MOV DPL, R1
    MOVX A, @DPTR
    MOV R5, A

    SWAP A
    ANL A, #0FH
    ACALL GET_LUT
    MOV DPTR, #PORTA
    MOVX @DPTR, A
    MOV DPTR, #PORTB
    MOV A, #0F7H
    MOVX @DPTR, A
    ACALL SHORT_DELAY
    MOV A, #0FFH
    MOVX @DPTR, A

    MOV A, R5
    ANL A, #0FH
    ACALL GET_LUT
    MOV DPTR, #PORTA
    MOVX @DPTR, A
    MOV DPTR, #PORTB
    MOV A, #0FBH
    MOVX @DPTR, A
    ACALL SHORT_DELAY
    MOV A, #0FFH
    MOVX @DPTR, A

```

```
    INC R1
    MOV DPH, R0
    MOV DPL, R1
    MOVX A, @DPTR
    MOV R5, A

    SWAP A
    ANL A, #0FH
    ACALL GET_LUT
    MOV DPTR, #PORTA
    MOVX @DPTR, A
    MOV DPTR, #PORTB
    MOV A, #0FDH
    MOVX @DPTR, A
    ACALL SHORT_DELAY
    MOV A, #0FFH
    MOVX @DPTR, A

    MOV A, R5
    ANL A, #0FH
    ACALL GET_LUT
    MOV DPTR, #PORTA
    MOVX @DPTR, A
    MOV DPTR, #PORTB
    MOV A, #0FEH
    MOVX @DPTR, A
    ACALL SHORT_DELAY
    MOV A, #0FFH
    MOVX @DPTR, A

    RET

GET_LUT:
    MOV DPTR, #LUT
    MOVC A, @A+DPTR
    RET

SHORT_DELAY:
    MOV R3, #5
SD_OUTER:
    MOV R4, #200
SD_INNER:
    DJNZ R4, SD_INNER
    DJNZ R3, SD_OUTER
    RET

DEBOUNCE:
    MOV R6, #255
DB1:
    DJNZ R6, DB1
    RET

LUT:
    DB 3FH, 06H, 5BH, 4FH, 66H, 6DH, 7DH, 07H, 7FH, 6FH
```

```
312
313 END
```

6. Results

Observed output summary:

- **Part (a):** Digits 0 to 9 displayed continuously in cyclic order on one 7-segment display.
- **Part (b):** Required non-linear sequence displayed correctly on two multiplexed displays.
- **Part (c):** Clock mode and stopwatch mode both functional; stopwatch start/stop and mode switching verified. In stopwatch mode, normal clock continued to update, confirming interrupt-driven concurrent timekeeping.

- **Video 1 (Display 0 to 9):** Watch on YouTube

- **Video 2 (Digital clock with stopwatch mode):** Watch on YouTube

7. Discussion

Soumadipta Majumder, 23EC30055

In Part (a), although the objective was simple, a clean 0–9 output on a 7-segment display was produced, and core concepts such as lookup-table based segment encoding, correct 8255 port initialization, and practical delay calibration were verified. It was also observed that even minor delay mismatches can cause inconsistent visual transitions, reinforcing the need for deterministic timing rather than rough software delay loops.

In Part (b), display multiplexing was studied in greater depth. Shared segment driving with independent digit-enable control was implemented, and the direct effect of refresh rate on brightness

and flicker was clearly observed. Because the required sequence was non-linear (00–06, 15–23, 29, 47–54), simple increment logic could not be used; therefore, explicit range-transition checks were incorporated. Confidence in register-level debugging was improved as number-splitting (tens/units) and LUT mapping were validated at each sequence boundary.

In Part (c), the key learning outcome was obtained from interrupt-driven concurrent timekeeping. A timer ISR was designed so that real-time clock registers were updated continuously, while stopwatch registers were updated conditionally through a RUN flag. The advantage of interrupt-based architecture over pure polling was thereby demonstrated for systems where multiple time-dependent tasks must execute simultaneously. Active-low button handling with debouncing was also implemented to reduce false triggers during mode switching, start/reset, and stop operations. Overall, understanding of embedded firmware structure, real-time constraints, and hardware-software co-design using 8051 with external peripheral ICs was significantly strengthened.

Jai Raj, 23EC10031

In Part (a), I understood the practical importance of lookup-table based coding for 7-segment displays. For showing decimal digits, the segment pattern for each digit (0–9) was stored in a table, and the corresponding code was sent directly to the display port. This made the implementation simple, repeatable, and easy to debug at the hardware interface level.

In Part (b), displaying two-digit numbers required additional data handling. Each number was divided by 10 so that quotient and remainder represented tens and units digits, respectively. These two values were stored in separate registers, mapped through the same lookup table, and then sent to the two displays individually during multiplex scanning. This part improved my understanding of number decomposition and register-level data flow in assembly programming.

My most important learning from this experiment was multiplexing. Initially, it appears that all displays are ON simultaneously, but in reality only one display is enabled at a time, and rapid switching between displays creates the visual illusion of continuous multi-digit output.

In Part (c), I learned how timer-interrupt based design is essential for reliable real-time behavior. The internal timer of the 8051 keeps running in the background and generates periodic overflow interrupts, which are used as the time base for updating clock and stopwatch registers. I also understood the practical role of mode switching and control keys: in clock mode the normal time is shown continuously, while in stopwatch mode start/reset and stop operations are handled without disturbing the main clock update. This helped me appreciate how interrupts enable concurrent tasks in embedded systems more effectively than only using polling delays.